Introduction to JAVA Programming Language

Lecture 2

Based on Slides of Dr. Norazah Yusof

Origins of the Java Language

- Java was created by Sun Microsystems team led by Patrick Naughton and James Gosling (1991)
- Originally named Oak (Gosling liked the look of an oak tree that was right outside his window at Sun)
- Intended to design a small computer language that could be used for consumer devices/appliances such as the cable TV switchboxes.
  - It is a difficult task because these devices do not have a lot of power or memory, the language had to be small and generate very tight code. Also, because different manufacturers may choose different central processing units (CPUs), it was important that the language not be tied to any single architecture.
- The team developed a two-step translation process to simplify the task of compiler writing for each class of appliances

Origins of the Java Language

- Significance of Java translation process
  - Writing a compiler (translation program) for each type of appliance processor would have been very costly
  - Instead, developed intermediate language that is the same for all types of processors : Java *byte-code*
  - Therefore, only a small, easy to write program was needed to translate byte-code into the machine code for each processor

Origins of the Java Language

- Patrick Naughton and Jonathan Payne at Sun Microsystems developed a Web browser that could run programs over the Internet (1994) and evolved into the HotJava browser.
  - Beginning of Java's connection to the Internet
  - To show off the power of applets, they made the browser capable of executing code inside web pages.
- Netscape Incorporated made its Web browser capable of running Java programs (1995)
  - Other companies follow suit

The Java Language

- Java is a full-featured, general-purpose programming language that is capable of developing robust mission-critical applications.

- Today it is used for developing stand alone applications, desktops, web programming, servers, and mobile devices.
  - Java can be run on the web browser called the *applets.*
  - Java can also be used to develop applications on the server side, called *Java servlets* or *Javaserver pages (JSP)*
  - Java can be used to develop applications for small hand-held devices, such as personal digital assistants and cell phones.

The Java Language Specification

- Java language specification defines the Java standard and the technical definition of the language that includes the syntax and semantics of the Java programming language.
  - Stable
- Java *application program interface* (API) contains predefined classes and interfaces for developing Java programs.
  - Still expanding

Java API

- Java 1.0 was introduced in 1995.
- December 1998, Sun announced the Java 2 platform – the brand that applies to current Java technology.
- There are 3 editions of the Java API:
  - Java 2 standard edition (J2SE)
    - client-side standalone applications or applets
  - Java 2 Enterprise Edition (J2EE)
    - server-side applications, such as Java servlets and JavaServer Pages
  - Java 2 Micro Edition (J2ME)
    - mobile devices, such as cell phones

Java Version History

Java Version	Release Year	Key Features & Enhancements
Java 1.0	1996	First stable version, AWT (Abstract Window Toolkit), Applets, Basic OOP support.
Java 1.1	1997	Inner classes, JDBC (Java Database Connectivity), JavaBeans, Reflection.
Java 2 (JDK 1.2)	1998	Swing, Collections Framework, JIT Compiler, Security enhancements.
Java 2 (JDK 1.3)	2000	Performance improvements, HotSpot JVM, JavaSound API.
Java 2 (JDK 1.4)	2002	Assertions, Regular Expressions, Logging API, NIO (New Input/Output).
Java 5 (JDK 1.5)	2004	Generics, Enhanced for-loop, Autoboxing/Unboxing, Varargs, Enum, Concurrency API.
Java 6	2006	Performance improvements, Scripting support, Compiler API, JDBC 4.0.

Java Version History

Java 7	2011	Try-with-resources, Diamond operator (`<>`), NIO.2, Fork/Join Framework.
Java 8	2014	Lambda Expressions, Streams API, Default & Static Methods in Interfaces, Optional class, Date-Time API.
Java 9	2017	Project Jigsaw (Modular System), JShell (REPL), HTTP Client (incubating).
Java 10	2018	Local-variable type inference (`var` keyword), Performance improvements.
Java 11 (LTS)	2018	Long-Term Support (LTS), HTTP Client API, ZGC (Z Garbage Collector), String Methods.
Java 12-16	2019-2021	Switch Expressions, Records (Preview), Pattern Matching, Sealed Classes.
Java 17 (LTS)	2021	Sealed Classes, Strong encapsulation, Foreign Function & Memory API (Preview).
Java 18-19	2022	Simple Web Server, Virtual Threads (Preview), Pattern Matching Enhancements.
Java 20-21 (LTS)	2023	Virtual Threads (Stable), Record Patterns, Sequenced Collections.

Java Development Tools

- Java development tool is a software that provides an integrated development environment (IDE) for rapidly developing Java programs.

- Other Java development tools on the market (besides JDK 5.0):
  - JBuilder by Borland
  - NetBeans Open Source by Sun
  - Eclipse Open Source by IBM

- Other useful tools:
  - Code warrior by Metrowerks
  - TextPad Editor
  - JCreator LE
  - JEdit
  - JGrasp
  - BlueJ
  - DrJava

Java Program

- Java program can be written in many ways:
  - Applications
  - Applets
  - Servlets

Java Applications

- A Java *applications* or "regular" Java programs are standalone programs that can be executed from any computer with a JVM.

- It is a class with a method named **`main`**
  - When a Java application program is run, the *run-time system* automatically invokes the method named **`main`**
  - All Java application programs start with the **`main`** method

Java Applets

- A Java *applet* (*little Java application*) is a Java program that is meant to be run from a Web browser
  - Can be run from a location on the Internet
  - Can also be run with an applet viewer program for debugging
  - Applets always use a windowing interface

Java Servlets

- A Java *servlets* are special kind of Java programs that run from a Web server to generate dynamic Web contents.

Objects and Methods in Java

- Java is an *object-oriented programming (OOP)* language
  - Programming methodology that views a program as consisting of *objects* that interact with one another by means of actions (called *methods*)
  - Objects of the same kind are said to have the same *type* or be in the same *class*

The Compiler and the Java Virtual Machine

- A programmer writes Java programming statements for a program.
- These statements are known as *source code (*.java)*
- A basic *text editor* can be used to edit and save a Java *source code file.*
- A *compiler* is a program that translates source code into an executable form *(*.class*)

The Compiler and the Java Virtual Machine

- A compiler is run using a source code file as input.
- Syntax errors that may be in the program will be discovered during compilation.
- *Syntax errors* are mistakes that the programmer has made that violate the rules of the programming language.
- The compiler creates another file that holds the translated instructions.

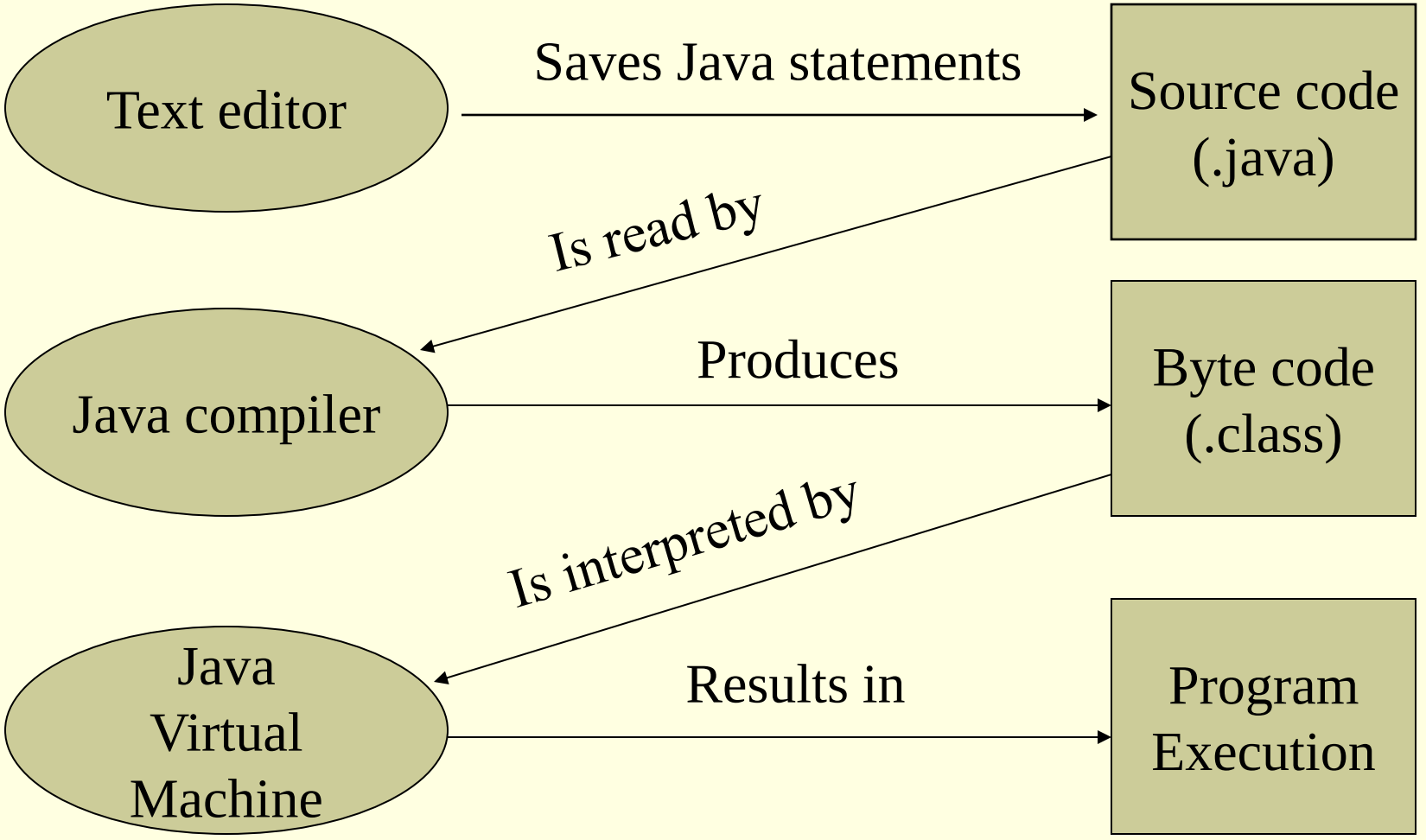
Byte-Code and the Java Virtual Machine (JVM)

- The compilers for most programming languages translate high-level programs directly into the machine language for a particular computer
  - Since different computers have different machine languages, a different compiler is needed for each one
- In contrast, the Java compiler translates Java programs into *byte-code (*.class)*, a machine language for a fictitious computer called the *Java Virtual Machine*
  - Once compiled to *byte-code*, a Java program can be used on any computer, making it very portable

The Compiler and the Java Virtual Machine

- Byte code files end with the *.class* file extension.
- The JVM is a program that *emulates* a micro-processor.
- The JVM executes instructions as they are read.
- JVM is often called an *interpreter.*
- Java is often referred to as an *interpreted language*.

Program Development Process



Portability

- *Portable* means that a program may be written on one type of computer and then run on a wide variety of computers, with little or no modification.

- Java byte code runs on the JVM and not on any particular CPU; therefore, compiled Java programs are highly portable.

- JVMs exist on many platforms:
  - Windows
  - Macintosh
  - Linux
  - Unix
  - BSD
  - Android
  - IOS
  - Etc.

Portability

- With most programming languages, portability is achieved by compiling a program for each CPU it will run on.

- Java provides an JVM for each platform so that programmers do not have to recompile for different platforms.

Portability

